Can AI Help Moderate Community Voice Platforms? Designing and Evaluating a Virtual Moderator

Abhijeet Malik

Supervised by Dr. Vikram Kamath Cannanure

chair for Societal Computing

Abstract

Community voice platforms in rural and resource-constrained regions rely on human content moderators to listen to phone-call questions, understand local context, link questions to expert answers, and escalate urgent messages [19]. These moderators are frequently overwhelmed: while most incoming messages are routine questions, serious issues such as mental health crises or health misinformation are often hidden among them, and moderators cannot listen to all messages in time. This thesis investigates whether small language models (SLMs) can assist such moderators by giving feedback on every message, connecting questions with expert answers, and identifying transcription or speech issues, so that the human moderator only needs to review the messages that require attention. We design and evaluate a *Virtual Moderator* built on privacy-preserving, resource-constrained SLMs for a healthcare moderation setting. The work is structured around three research questions: (RQ1) developing and validating a Virtual Moderator against human expert consensus, (RQ2) using the validated Virtual Moderator to compare candidate SLMs for deployment, and (RQ3) analyzing how speech-based interactions influence model performance. Initial results show that small models in the 1B–4B parameter range achieve moderate agreement with human moderators, and error analysis on speech-based interactions reveals that misinterpretation and lack of answers are the leading causes of model failure.

1 Introduction

Voice-based community platforms provide an accessible means of information exchange for populations for whom conventional digital services may be difficult to access. In particular, Interactive Voice Response (IVR) systems allow users to access and contribute information using ordinary mobile phones, without requiring a smartphone or an Internet connection. Such systems have therefore been used in low-income and low-literacy communities for applications ranging from citizen journalism and grievance reporting to agriculture, livelihood support, and other forms of community information exchange Khullar et al. [19].

A prominent example is *Mobile Vaani*, a network of voice-based community platforms operated by the Indian social enterprise Gram Vaani. Users can listen to audio content and contribute their own voice recordings, which are subsequently reviewed before being made available through the platform. Content moderation is therefore a central component of the system. Human moderators are responsible for tasks such as checking audio quality and relevance, applying editorial guidelines, annotating recordings, and producing or reviewing transcriptions . At the time of the study by Khullar et al. [19], approximately fifteen full-time moderators were responsible for content operations across more than 40 voice-based forums, collectively receiving roughly 1,000 voice recordings per day.

This dependence on human moderation introduces an important scalability challenge. As voice-based platforms grow, the amount of moderation work grows with them, while increasing the size of the moderation team is constrained by both financial and organisational resources.

Moreover, many moderation operations are repetitive or routine, potentially diverting moderator attention from cases that require more careful human judgement . Khullar et al. [19] investigated this problem by introducing AI-based tools for tasks such as detecting empty or noisy recordings, automatically transcribing speech, and extracting metadata from transcripts. Their results showed that automation can substantially reduce moderation time, but also demonstrated that it cannot fully replace human moderators. Instead, AI and human moderators can play complementary roles, with automation handling suitable routine operations while human judgement remains important for more complex decisions.

The present thesis builds on this idea of AI-assisted moderation but addresses a different and more semantically demanding part of the moderation process. Rather than focusing primarily on audio preprocessing, transcription, or metadata extraction, this work investigates whether language models can assist with evaluating the quality of responses provided to users. This is particularly important in the healthcare setting considered in this thesis, where questions may concern sensitive topics and an incorrect, irrelevant, or contextually inappropriate response can have considerably greater consequences than an error in a routine content-processing task hewitt2025health [3].

The empirical work in this thesis is based on three datasets provided by Gram Vaani. These datasets contain question–answer interactions from the target deployment setting and make it possible to study moderation under realistic conditions. For each question–answer pair, the central moderation task is to determine whether the generated answer is *accurate* and whether it is *contextually relevant* to the user's question. Performing these evaluations manually at scale is demanding, since a human moderator must carefully interpret both the question and the corresponding answer before making a judgement.

This motivates the development of a *Virtual Moderator*: an AI-based moderation system designed to approximate these human moderation judgements. The Virtual Moderator evaluates question–answer pairs along two dimensions, *Accuracy* and *Context & Relevance*, and is validated against annotations provided by human experts. The objective is not to remove the human moderator from the decision process. Instead, the system is intended to support a human-in-the-loop moderation workflow in which automated evaluation can reduce repetitive review effort and help direct human attention towards cases that require closer inspection. This follows the broader observation of Khullar et al. [19] that AI-based moderation is most useful as a means of augmenting, rather than completely replacing, human moderators.

Large language models (LLMs) provide strong language-understanding and evaluation capabilities, but deploying large cloud-based models can be undesirable in the target setting due to computational requirements, inference cost, latency, and the sensitive nature of healthcare-related data. This thesis therefore places particular emphasis on *small language models* (SLMs). Their smaller computational footprint makes local deployment more feasible and creates the possibility of performing moderation without continuously sending sensitive user interactions to external cloud services. However, this advantage is useful only if smaller models can provide moderation judgements that are sufficiently consistent with those of human experts.

The thesis initially perform the inference task for the models freely available on the cloud platform such as Huggingface and ollama . From these platforms , the models listed there (small language models) can be locally downloaded (as they are small in size) and used . Thesis explored a large number of models on the dataset of 100 questions-answer pairs provided by the Gram Vaani . The most robust and consistent model is being choosen. The model choosen in the inference phase is being further fine tuned for the downstream task . In fine tuning , a pre-trained model is taken and trained further on a specific , smaller dataset to make it an expert on one particular task (First time fine tuning had made the impact was in 2014, when ImageNet competitions proved that deep convolutional neural networks (CNNs) were highly capable at processing images, developers began saving pre-trained weights. They realized they could take a model trained to recognize millions of general items and "fine-tune" its last few layers to identify highly specific objects .(Modern language fine-tuning exploded with papers like ULMFiT, OpenAI GPT, and Google's BERT. Instead of training language networks from scratch, researchers could take a model that already understood

general language structures and adapt it to target tasks (like sentiment analysis or legal document extraction).

Building an AI model completely from scratch costs millions of dollars and requires massive datasets. Fine-tuning solves this problem by modifying a model that is already smart.Domain Specialization: A base AI model knows general conversational rules but lacks deep specialized expertise. Fine-tuning forces it to master legal jargon, complex medical notes, or proprietary corporate data.Consistent Behavior Formatting: If you need an AI to reliably output information in a strict format—like a standardized JSON object—prompting alone can occasionally fail. Fine-tuning deeply instills these operational rules into the model's weights.Efficiency and Lower Latency: Instead of typing long instructions and examples into every single prompt (few-shot prompting), fine-tuning embeds that knowledge into the core architecture. This reduces prompt sizes, leading to cheaper API costs and faster response times.

There are also number of types of fine tuning :

- By Parameter Adjustment (The Mechanics): By parameter adjustment we can do fine tuninig in two ways , Fully Fine Tuning (FFT) Every single weight and parameter in the model is updated. It delivers excellent accuracy but demands high computational power and risks a flaw known as "catastrophic forgetting"—where the model gains a new skill but completely loses its original capabilities.(Perameter Efficient Fine Tuning (PEFT) - The base model's original weights are frozen. Instead, a tiny subset of new parameters is added and trained. one such type of this fine tuning is Low Rank Adaptation (LoRA) . It injects small, manageable math matrices into the model's layers to absorb the new information efficiently without altering the core structure. This is the technique this thesis uses in order to do the fine tuning of the choosen model . It perticularly make very small number of weight updates and keeps the initially layers of the model original . (

- By Training Method (The Objective) : The model is trained on distinct prompt-and-response pairs (e.g., Prompt: "Fix this grammar mistake" : "[Corrected Text]"). This teaches the AI how to behave like a helpful assistant.Reinforcement Learning from Human Feedback (RLHF): Humans rate and rank different outputs generated by the AI. The model uses this feedback to align its judgment with human preferences, learning safety boundaries and stylistic choices.

A further challenge arises from the voice-based nature of the platform. In a real deployment, a user's input does not necessarily originate as clean written text. It may contain transcription errors, incomplete utterances, background noise, colloquial expressions, or other characteristics of spontaneous speech. Consequently, evaluating a moderation system only on clean textual input may overestimate its effectiveness in practice[5]. This thesis therefore considers not only model-level moderation performance but also the influence of speech-based interactions on the overall evaluation pipeline.

1.1 Motivating Scenario

Amina is one of the human moderators responsible for reviewing a large number of recorded calls every day. For each user question, she also examines the answer generated by the knowledge-based system, knowledge based model. Her task is to determine whether the generated answer is factually correct, whether it is relevant to the user's question and context, and whether the tone is sufficiently clear, sensitive, and reassuring for the user to understand and trust.

This is demanding and attention-intensive work. Repeatedly listening to calls and evaluating generated answers can lead to fatigue and loss of concentration. However, even a brief lapse in attention may have serious consequences, particularly in sensitive mental-health cases. For example, a teenager may be experiencing relationship difficulties, thoughts of self-harm, violence at home, loneliness, or severe anxiety related to examinations and academic pressure. Such situations require careful interpretation and responsible guidance. An incorrect, irrelevant, or insensitive answer generated by Sarvam AI could worsen the user's situation or prevent an urgent case from receiving timely attention.

As a human moderator, Amina naturally becomes tired after reviewing many interactions. This thesis therefore aims to reduce her workload by developing a Virtual Moderator that can automatically evaluate the generated answers. The system assesses whether an answer is accurate, contextually relevant, and appropriately expressed. Rather than replacing Amina, the Virtual Moderator is intended to support her by identifying interactions that require human attention, allowing her to focus on the most sensitive and high-risk cases.

1.2 Research Questions

Based on these challenges, this thesis addresses the following research questions:

- **RQ1:** How can a Virtual Moderator be developed and validated to evaluate responses in a healthcare context?
- **RQ2:** How do different AI models compare when evaluated using the Virtual Moderator developed in RQ1?
- **RQ3:** How do speech-based interactions influence AI model performance, and what opportunities and challenges emerge in the target context?

Together, these research questions progress from the development and validation of an automated moderation mechanism, to its use as a scalable evaluation tool, and finally to an examination of the practical challenges introduced by real-world voice interactions. In this way, the thesis investigates whether resource-efficient language models can provide meaningful assistance to human moderators while preserving human oversight for decisions that require contextual understanding and careful judgement.

In this Thesis , first we start with with inferencing different small language models availabe on Hugging Face and Ollama . we tried to check for the models which are of not very large in the size otherwise it would be difficult to host them locally because of resources constraints .

Gemma models are remarkably good for the task of data moderation in the health care system . there high capability of tuning towads the specific field make them perfect for annotation . They can be dfepoloyed on the local cloud or on-premises becuase of their small sizes .

2 Related Work

The growing deployment of large language models (LLMs) in conversational systems has increased the need for reliable mechanisms that can detect harmful prompts, unsafe model responses, policy violations, and inappropriate refusals. Earlier content-moderation systems were commonly developed as task-specific classifiers for categories such as toxicity, hate speech, harassment, and sexually explicit content. However, the open-ended nature of LLM interactions introduces more complex risks, including harmful instructions, jailbreak attempts, context-dependent abuse, medical misinformation, and unsafe responses that may not contain explicitly toxic language(Consequently, recent research has shifted toward dedicated guard models, multilingual safety datasets, domain-specific risk taxonomies, and LLM-based evaluators. This section reviews these developments, with particular attention to efficient small language models, multilingual moderation, health-related risk detection, and the reliability of automated evaluation.

2.1 LLM-Based Content Moderation and Guard Models

One of the foundational works in modern LLM safeguarding is Llama Guard, introduced by **Inan et al.** [5]. Unlike conventional toxicity classifiers that assign labels based on a fixed dataset, Llama Guard treats moderation as an instruction-following task governed by a configurable safety taxonomy. It can classify both user prompts and model-generated responses, thereby supporting input and output moderation within conversational applications. This policy-conditioned approach is important because the definition of unsafe content may vary across applications, organisations, and deployment contexts. Subsequent guard models expanded the range of safety-related tasks. **WildGuard** [6] provides a unified framework for identifying harmful user intentions, detecting harmful model responses, and determining whether a model has refused to answer. Its associated dataset includes direct and adversarial interactions across 13 risk categories. WildGuard is therefore relevant not only as a moderation model but also as an automated evaluator for

safety-alignment experiments. In particular, its explicit treatment of refusal behaviour helps distinguish between unsafe compliance and appropriate rejection of harmful requests. Similarly, **ShieldGemma** [7] presents a family of content-moderation models based on Gemma 2. The models evaluate prompts and responses against safety policies related to sexually explicit material, dangerous content, harassment, and hate speech. ShieldGemma demonstrates the increasing use of generative, instruction-tuned models as policy-aware safety classifiers rather than relying exclusively on traditional encoder-based architectures. However, its core policies remain relatively broad and may not capture domain-specific harms, such as subtle medical misinformation or unsafe clinical recommendations. Taxonomy design has also become a central component of guardrail development. **AEGIS2.0** [8] introduces an adaptable safety taxonomy consisting of broad hazard categories and more fine-grained subcategories. It combines human annotation with judgments produced by multiple LLMs to construct a dataset of human–LLM interactions for guardrail development. This approach demonstrates that safety moderation cannot be reduced to a universal binary distinction between safe and unsafe content. Instead, useful moderation systems require explicit definitions of risks and sufficient flexibility to support different policies and application domains. Collectively, these studies represent a transition from narrow toxicity detection toward general-purpose conversational safeguards. Nevertheless, general guard models introduce practical limitations. Their broad taxonomies may not represent risks that are specific to a particular domain, while the computational requirements of large generative guards can restrict their use in real-time, privacy-sensitive, or resource-constrained environments.

2.2 Small Language Models for Efficient Moderation

Although large guard models offer strong general-language capabilities, their inference costs, memory requirements, latency, and dependence on external services may make them unsuitable for some moderation applications. These limitations have motivated research into small language models (SLMs) that can be fine-tuned for specialised safety-classification tasks. Zhan et al. introduced **SLM-Mod** [1] to investigate whether specialised SLMs can provide an effective alternative to substantially larger LLMs for community-specific content moderation. The authors fine-tuned models with fewer than 15 billion parameters using approximately 150,000 comments collected from 15 Reddit communities. Their experiments showed that the fine-tuned SLMs outperformed zero-shot LLMs by an average of 11.5 percentage points in accuracy and 25.7 percentage points in recall. Few-shot prompting produced only limited improvements for the larger models. These findings suggest that task-specific supervision may be more important than model scale for well-defined moderation tasks. SLM-Mod also demonstrates the importance of contextualising moderation policies. Content that is acceptable in one online community may violate the rules or expectations of another. Fine-tuned SLMs can learn such local distinctions while avoiding the expense of repeatedly querying a general-purpose LLM. However, the study is primarily concerned with Reddit community moderation. Social-media moderation differs from safety detection in health-related conversations, where risks may arise from inaccurate medical recommendations, delayed treatment, unsuitable medication guidance, failure to recognise emergencies, or inappropriate handling of sensitive personal information. **Hewitt et al.** [3] extend the SLM-based moderation problem into the healthcare domain by proposing a health-focused AI risk taxonomy developed through a literature review and co-design with healthcare professionals. The authors evaluate six SLMs using their proposed taxonomy and other safety taxonomies. Their findings show that Granite Guardian, an eight-billion-parameter model, was able to adapt to the health-focused taxonomy without additional fine-tuning and obtained an accuracy of 75%. This result provides evidence that compact models can support domain-specific safety detection and may be suitable for local deployment in settings where privacy and computational efficiency are important. These studies indicate that smaller, specialised models can be competitive with larger general-purpose models when the target risks and moderation policies are clearly defined. However, further investigation is required to determine whether these advantages generalise across languages, health-risk categories, prompt styles, and datasets that differ from the model's training distribution.

2.3 Parameter-Efficient Adaptation with LoRA and QLoRA

Fine-tuning an entire language model can require substantial GPU memory and computational resources, even when the target task is relatively specialised. Hu et al. [17] introduced Low-Rank Adaptation (LoRA) as a parameter-efficient alternative to full fine-tuning. Instead of updating all parameters of the pretrained model, LoRA keeps the original model weights frozen and introduces small trainable low-rank matrices into selected Transformer layers.

2.3.1 Mathematical Formulation of LoRA

Consider a pretrained linear transformation with weight matrix

$$\mathbf{W}_0 \in \mathbb{R}^{d\times k},$$

where $\mathbf{W}_0$ represents the original pretrained weights. Under conventional full fine-tuning, the complete matrix would be updated during training,

$$\mathbf{W} = \mathbf{W}_0 + \Delta\mathbf{W},$$

where $\Delta\mathbf{W}$ has the same dimensionality as $\mathbf{W}_0$ and therefore contains dk trainable parameters.

LoRA assumes that the task-specific weight update can be represented by a matrix of substantially lower rank. Instead of directly learning $\Delta\mathbf{W}$, it is decomposed as

$$\Delta\mathbf{W} = \mathbf{BA},$$

with

$$\mathbf{A} \in \mathbb{R}^{r\times k}, \qquad \mathbf{B} \in \mathbb{R}^{d\times r},$$

where

$$r \ll \min(d, k)$$

is the LoRA rank. The effective weight matrix during inference is therefore

$$\mathbf{W} = \mathbf{W}_0 + \mathbf{BA}.$$

For an input representation $\mathbf{x}$, the corresponding transformation becomes

$$\mathbf{h} = \mathbf{W}_0\mathbf{x} + \mathbf{BAx}.$$

In practice, LoRA introduces an additional scaling parameter α, giving

$$\mathbf{h} = \mathbf{W}_0\mathbf{x} + \frac{\alpha}{r}\mathbf{BAx}.$$

Here, $\mathbf{W}_0$ remains frozen throughout training, while only $\mathbf{A}$ and $\mathbf{B}$ are updated by gradient descent.

The reduction in the number of trainable parameters can be seen directly. Full fine-tuning of the original matrix requires

$$N_{\text{full}} = dk$$

trainable parameters, whereas LoRA requires only

$$N_{\text{LoRA}} = r(k + d).$$

The ratio of trainable LoRA parameters to full parameters is therefore

$$\frac{N_{\text{LoRA}}}{N_{\text{full}}} = \frac{r(k + d)}{dk}.$$

Because r is typically much smaller than either d or k, only a small fraction of the parameters of the original transformation need to be optimised.

For example, if

$$d = k = 4096$$

and a LoRA rank of

$$r = 16$$

is used, full fine-tuning of the matrix would require

$$4096 \times 4096 = 16{,}777{,}216$$

trainable parameters. In contrast, LoRA requires only

$$16(4096 + 4096) = 131{,}072$$

trainable parameters for the corresponding update. This is less than one percent of the parameters required to update the complete matrix.

In Transformer-based language models, LoRA is commonly applied to the projection matrices used in the attention mechanism. In this work, adapters are introduced into the query, key, value, and output projections,

$$\mathbf{W}_Q,\ \mathbf{W}_K,\ \mathbf{W}_V,\ \mathbf{W}_O,$$

while the original pretrained parameters remain fixed. For example, the query projection becomes

$$\mathbf{W}'_Q = \mathbf{W}_Q + \frac{\alpha}{r}\mathbf{B}_Q\mathbf{A}_Q.$$

The same principle is applied independently to the other selected projection matrices.

2.3.2 QLoRA

Dettmers et al. [18] extended this approach with Quantized Low-Rank Adaptation (QLoRA). QLoRA further reduces memory consumption by storing the frozen pretrained model in a low-precision quantized representation while keeping the LoRA adapters trainable.

Let

$$Q(\mathbf{W}_0)$$

denote the quantized representation of the pretrained weight matrix. During computation, the quantized weights are interpreted using an appropriate dequantization operation,

$$\widetilde{\mathbf{W}}_0 = Q^{-1}(Q(\mathbf{W}_0)).$$

The adapted transformation can then be expressed as

$$\mathbf{h} = \widetilde{\mathbf{W}}_0\mathbf{x} + \frac{\alpha}{r}\mathbf{B}\mathbf{A}\mathbf{x}.$$

The quantized base-model parameters remain frozen. Gradients are propagated through the model in order to optimise only the LoRA matrices $\mathbf{A}$ and $\mathbf{B}$:

$$\nabla_{\mathbf{W}_0}\mathcal{L} = 0,$$

while

$$\nabla_{\mathbf{A}}\mathcal{L} \neq 0, \qquad \nabla_{\mathbf{B}}\mathcal{L} \neq 0.$$

Thus, the optimisation problem can be written conceptually as

$$\mathbf{A}^*, \mathbf{B}^* = \arg\min_{\mathbf{A},\mathbf{B}} \mathcal{L}\left(f_{\mathbf{W}_0,\mathbf{A},\mathbf{B}}(x), y\right),$$

where x denotes the moderation input, y denotes the target moderation labels, and $\mathcal{L}$ is the training loss.

For the Virtual Moderator, the target y consists of the two teacher-generated moderation decisions,

$$y = \left(y_{\text{Accuracy}}, y_{\text{Context\&Relevance}}\right),$$

with

$$y_{\text{Accuracy}}, y_{\text{Context\&Relevance}} \in \{0, 1, 2\}.$$

Consequently, the pretrained language model provides the underlying linguistic and reasoning capabilities, while the LoRA adapters learn the task-specific mapping from a healthcare question–answer pair to the required moderation labels.

In the implementation used in this work, the base language model is loaded using 4-bit quantization and LoRA adapters are trained on selected attention projection layers. A rank of $r = 16, 8, 4$ and scaling factor $\alpha = 32, 16, 8$ are used. The resulting architecture can be summarised as

$$\boxed{\text{Frozen 4-bit Base Model} + \text{Trainable LoRA Adapters}}$$

rather than updating the complete multi-billion-parameter model.

This formulation is particularly suitable for the iterative supervision pipeline used in this thesis. At each training-set size, the same pretrained base model can be retained while a lightweight LoRA adapter is learned from the available Claude-generated supervision. This makes it possible to compare adaptation with increasing numbers of labelled examples without repeatedly performing computationally expensive full-model fine-tuning.

2.4 Multilingual and Culturally Grounded Safety Moderation

A significant limitation of existing safety research is its concentration on English-language data. Safety alignment learned predominantly from English examples may not transfer consistently to other languages, particularly languages with fewer training resources or different cultural expressions of harmful content. Wang et al. introduced **XSafety** [9], one of the earliest systematic multilingual safety benchmarks for LLMs. XSafety covers 14 safety scenarios across ten languages, including Hindi. The evaluation demonstrated that the tested LLMs generated substantially more unsafe responses for non-English prompts than for equivalent English prompts. This finding indicates that general multilingual capability does not necessarily imply multilingual safety and that safety alignment can weaken when interactions are conducted in languages that are less represented in training data. The distinction between linguistic translation and cultural adaptation is further examined in **RTP-LX** [10]. RTP-LX contains human-transcreated and human-annotated toxic prompts and outputs in 28 languages. Instead of depending entirely on literal machine translation, the dataset uses participatory design and includes culturally specific forms of toxicity. Its evaluation shows that models may achieve apparently acceptable accuracy while still exhibiting weak agreement with human judgments, particularly for context-dependent harms, microaggressions, and subtle bias. The work therefore highlights the importance of considering annotation agreement and class distribution rather than relying only on aggregate accuracy. Building on multilingual safety datasets, Kumar et al. proposed **PolyGuard** [2], a multilingual safety-moderation model covering 17 languages, including Hindi (The language our dataset and downsteam task based on). PolyGuard

is trained on PolyGuardMix, which contains approximately 1.91 million multilingual examples. The authors also introduce PolyGuardPrompts, an evaluation benchmark containing approximately 29,000 samples. The datasets label prompt harmfulness, response harmfulness, and response refusal, allowing PolyGuard to address a set of moderation tasks similar to WildGuard in a multilingual setting. PolyGuard reportedly improves over existing open and commercial safety classifiers across several safety and toxicity evaluations. PolyGuard represents an important step toward scalable multilingual moderation. Nevertheless, part of its data is produced through human-verified machine translation of English safety examples. Translation-based scaling improves language coverage, but translated examples may preserve the assumptions and risk definitions of the source language. They may fail to capture native expressions, code-switching, region-specific medical practices, culturally sensitive health topics, or harmful content that depends on local social context. Multilingual moderation should therefore be evaluated not only on translated test sets but also on naturally occurring and culturally grounded examples. Overall, multilingual safety research shows that performance can vary considerably across languages even when the underlying harmful intent is equivalent. It also suggests that low-resource and culturally diverse languages require language-aware evaluation rather than being treated as direct translations of English.

2.5 Safety Benchmarks and Domain-Specific Risk Taxonomies

The reliability of a moderation model depends heavily on the datasets and taxonomies used to train and evaluate it. Many earlier toxicity benchmarks were constructed from social-media posts and comments. However, user interactions with conversational AI systems differ from conventional social-media communication in their structure, purpose, and context. **ToxicChat** [11] addresses this limitation by introducing a benchmark derived from real user queries submitted to an open-source chatbot. The dataset contains approximately 10,000 user–AI conversations and includes nuanced examples that are difficult for models trained on conventional social-media toxicity datasets. Experiments reported in the study reveal a meaningful domain shift between social-media toxicity detection and moderation of user–AI interactions. This finding supports the use of conversational data when evaluating guard models intended for chatbot deployment. Broader evaluation frameworks have also attempted to organise safety risks systematically. **SALAD-Bench** [12] introduces a large-scale benchmark with a three-level hierarchical taxonomy. It includes conventional safety questions as well as examples modified using adversarial attacks and defence strategies. SALAD-Bench also proposes MD-Judge, an LLM-based evaluator that assesses question–answer pairs according to multiple safety dimensions. The hierarchical structure is useful because it enables performance to be examined at different levels of granularity rather than reporting only an overall safe-versus-unsafe score. General safety taxonomies, however, may not sufficiently represent risks encountered in healthcare applications. A health-related response can be harmful without containing hate speech, violence, explicit content, or illegal instructions. Examples include confidently presenting incorrect medical information, recommending an unsuitable treatment, failing to advise emergency assistance, contradicting established clinical evidence, or encouraging a user to discontinue prescribed medication(**MedSafetyBench** [13] addresses this problem by defining medical safety with reference to principles of medical ethics and constructing a benchmark for evaluating medical LLMs. Its evaluation indicates that strong performance on medical question-answering tasks does not guarantee safe behaviour. The study also shows that safety-oriented fine-tuning can improve medical safety while retaining general medical performance. MedSafetyBench and the health-focused taxonomy proposed by **Hewitt et al.** [3] therefore complement general guardrail taxonomies by demonstrating that safety definitions should account for the requirements of the target domain. An additional challenge in health-related moderation is avoiding excessive blocking. Terms related to suicide, substance use, medication, abuse, sexual health, or serious illness may occur in legitimate requests for information or assistance. A moderator that relies primarily on the presence of sensitive vocabulary may incorrectly classify beneficial health conversations as unsafe. Consequently, a suitable health-moderation system must balance the recall of genuinely harmful content against the false-positive classification of legitimate, educational, or help-seeking content(

2.6 LLMs as Automated Judges

Human evaluation is often considered the most reliable method for assessing open-ended language-model outputs, but it is expensive, time-consuming, and difficult to scale. LLM-as-a-judge methods have therefore become increasingly common in model evaluation. **Zheng et al.** [4] introduced MT-Bench and Chatbot Arena to investigate whether strong LLMs could approximate human preferences when comparing chatbot responses. Their results showed that an LLM judge could achieve more than 80% agreement with human preferences. The study also identified important limitations, including position bias, verbosity bias, self-enhancement bias, and limited reasoning ability. These observations established both the potential and the risks of using LLMs as automatic evaluators. Later work has examined the reliability of judge models more directly. **JudgeBench** [14] evaluates LLM judges using difficult response pairs from knowledge, reasoning, mathematics, and programming tasks where preference labels are based on more objective correctness criteria. The benchmark shows that several otherwise capable models struggle to distinguish between subtly different responses. This suggests that agreement with general human preference does not necessarily establish the reliability of a judge for specialised evaluation tasks. Safety evaluation presents additional difficulties because judgments may depend on policy interpretation and contextual reasoning. Chen and **Goldfarb-Tarrant** [15] evaluate 11 LLM judges across safety domains and examine their self-consistency, agreement with human annotations, and sensitivity to superficial response features. Their results show that safety judgments can be influenced by artefacts such as apologetic phrasing and response verbosity. A response may therefore appear safer to an automated evaluator because it uses cautious language, even when its substantive content remains harmful. The reliability problem becomes especially important in the medical domain. **Diekmann et al.** [16] compare LLM-based medical safety judgments with human annotations across several qualitative criteria. Although the evaluated judges perform comparatively well on more objective properties, such as grammaticality and consistency with established scientific knowledge, they experience greater difficulty with subjective and context-dependent safety dimensions. This result supports the use of expert or human-validated annotations when assessing health-related moderation systems. LLM judges remain useful for scalable experimentation, particularly when comparing many model configurations. However, their outputs should not automatically be treated as ground truth. Robust evaluation practices may include the use of explicit rubrics, randomisation of response order, repeated evaluations, multiple judges, reporting of inter-rater agreement, and validation against a human-annotated subset.

A further step toward scalable evaluation is the development of smaller models trained specifically to judge the responses of other language models. In **Kim et al.** [20] *introduces an evaluator Prometheus 2. An Open Source Language Model Specialized in Evaluating Other Language Models* that can assess individual responses or compare pairs of responses using user-defined evaluation criteria. They test its judgments against both human assessments and those of proprietary language-model judges, reporting strong agreement across multiple evaluation benchmarks. This is relevant to the present study because the Virtual Moderator also assigns criterion-specific judgments to question–answer pairs, rather than producing answers itself. Its Accuracy and Context and Relevance labels are then used to assess responses generated by candidate models. However, the performance reported for Prometheus 2 does not establish that a different evaluator will be reliable for Hindi healthcare interactions. The Virtual Moderator must therefore be validated against the available human expert labels before its judgments are used to compare answer-generating models.

2.7 Error Analysis in Speech-Based Healthcare Chatbots

Overall accuracy indicates how often a system succeeds, but it does not explain why an interaction fails. In Integrated Taxonomy of Errors in Chat-Oriented Dialogue Systems, **Higashinaka et al.**[21] develop a structured way to categorise conversational errors and show that such categories can be annotated with reasonable consistency. This approach is useful when examining a chatbot because an incorrect interaction may result from misunderstanding the question, producing an incomplete answer, losing relevant context, or generating factually incorrect information. Identifying

these patterns provides a clearer account of system limitations than reporting a single error rate.

Speech-based healthcare interactions add another source of failure before a chatbot or moderator evaluates the question. In Classification of Failures in the Perception of Conversational Agents (CAs) and Their Implications on Patient Safety,**Aftab et al.**[22] distinguish failures in speech recognition from failures in language understanding and relate these to potential patient-safety risks. More recently, WER is Unaware **ellis2026wer**[23]: Assessing How ASR Errors Distort Clinical Understanding in Patient Facing Dialogue shows that the number of transcription errors alone does not reliably indicate their clinical impact: a small change to a medically important utterance may matter more than several harmless word substitutions. These findings motivate examining both the type of error and the stage at which it arises.

For the present study, this distinction is necessary when analysing spoken healthcare questions and their generated answers. An incorrect response may originate from a distorted transcript, from the answer-generating model's interpretation or medical content, or from an incorrect judgment by the Virtual Moderator. Categorising minor and substantial errors, and examining their distribution across health topics and speakers, helps identify where the system is unreliable and where human review is most needed. Attribution to transcription should be made only when the available audio or other evidence supports it.

2.8 Summary and Research Gap

Existing research has made substantial progress in developing general-purpose guard models, efficient SLM-based classifiers, multilingual safety benchmarks, domain-specific risk taxonomies, and scalable LLM-based evaluation. Nevertheless, these research directions have often been examined separately. General-purpose safeguards such as Llama Guard, WildGuard, ShieldGemma, and AEGIS2.0 provide broad coverage of conversational risks but are not specifically designed to distinguish between safe and unsafe health-related communication. Research on SLMs demonstrates that specialised compact models can outperform much larger zero-shot models, although much of this evidence is based on social-media or community-moderation data. Health-safety benchmarks and taxonomies capture domain-specific risks, but multilingual variation and deployment efficiency remain comparatively underexplored. Multilingual systems such as PolyGuard improve cross-lingual coverage, yet multilingual safety data continue to depend partly on translated English examples and may not adequately represent culturally grounded health risks. Finally, LLM-based judges provide scalable evaluation but may introduce inconsistent or artefact-sensitive judgments. These limitations motivate research at the intersection of efficient SLM-based moderation, multilingual safety, health-focused risk classification, and reliable evaluation. In particular, there remains a need to determine whether compact language models can identify fine-grained health-related risks across languages such as English and Hindi while maintaining a suitable balance between harmful-content detection and the preservation of legitimate health discussions. Such evaluation should consider not only overall accuracy but also class-specific precision, recall, F1-score, false-positive behaviour, cross-language generalisation, and agreement with human or expert judgments. This combined perspective can support the development of moderation systems that are computationally efficient, context-sensitive, linguistically inclusive, and appropriate for safety-critical health applications.

3 Dataset

All the following datadset are made available by Gram Vaani . These are all the real question answers asked by Users in the region . These questions are being fed to the Knowledge based model , Sarvam AI. The answers generated are being reported as answer . we have the sanitized version of the Queries and also the original question asked by the Users . There is no sanitized data used in the making of the Virtual Moderator . Three distinct datasets support the three research questions of this thesis.

3.1 RQ1 Dataset

For the initial validation and benchmarking of the Virtual Moderator, a fixed human-annotated healthcare moderation dataset of 100 question–answer pairs is used. Each instance is labelled by

human moderators for two dimensions: *Accuracy* and *Context & Relevance*, both on a three-level scale from 0 to 2. This 100-example dataset is retained as the fixed validation set throughout the subsequent adaptation experiments and is never used for LoRA parameter updates.

A larger pool of 1,660 previously unlabelled question–answer pairs is available for iterative adaptation. It contains 296 mental-health queries and 1,364 sexual-health queries. Before adaptation begins, 150 examples are separated using a stratified shuffle procedure and reserved as a final held-out test set. These examples are not used for prompt optimization, training, model selection, or intermediate error analysis.

The remaining examples form the iterative supervision pool. Successive batches of 100 examples are labelled by a Claude-based external teacher model using the same Accuracy and Context & Relevance rubric. These teacher-generated labels are treated as pseudo-labels for prompt optimization and LoRA/QLoRA instruction fine-tuning of the selected student model.

3.2 RQ2 Dataset

RQ2 uses the larger healthcare question–answer collection to compare candidate models at scale after the Virtual Moderator has been established. The dataset contains naturally occurring Hindi user questions and the corresponding answers produced in the target application setting, together with translated versions where available. The original Hindi text is used for the moderation experiments.

The dataset does not initially contain Accuracy or Context & Relevance labels for all examples. During RQ1 adaptation, labels for the training batches are generated by the Claude teacher model. The fixed 100-example human-labelled validation set and the 150-example held-out final test set remain separate from these iterative training batches.

3.3 RQ3 Dataset

For analyzing speech-based interactions, a dataset of 140 queries collected from 10 users is used, with each user asking 14 questions to the model, including some repeated questions. This dataset enables both category-wise (mental health vs. sexual health) and user-wise error analysis. This dataset also contains the labels for correct , incorrect and minor error for the answers generated by Sarvam AI . These labels are given by the Experts working for the NGO

4 Methodology

This section describes the construction and validation of the Virtual Moderator in RQ1, the comparison of candidate models in RQ2, and the error analysis used for speech-based interactions in RQ3. RQ1 consists of two stages: an initial benchmark against human expert labels, followed by LLM-assisted iterative supervision of selected resource-efficient models.

4.1 RQ1: Building and Validating the Virtual Moderator

RQ1 investigates whether a Small Language Model (SLM) can act as a *Virtual Moderator* whose decisions are sufficiently aligned with those of human healthcare experts. The human-annotated healthcare moderation dataset is used as the reference standard. Each data instance contains a Hindi user question, its corresponding expert answer, and expert-assigned labels for two moderation dimensions:

1. **Accuracy:** whether the answer is factually correct and appropriate for the healthcare question; and
2. **Context and Relevance:** whether the answer addresses the user's question and is suitable within the context of the interaction.

Both dimensions are represented on the scale $\{0, 1, 2\}$, where 0 denotes the lowest assessment, 1 denotes a partially satisfactory assessment, and 2 denotes the highest assessment.

The original Hindi questions and answers are used directly during inference. They are not translated into English. This preserves the linguistic characteristics of the original interactions, including informal expressions, code-switching, regional vocabulary, and possible speech-transcription errors.

4.1.1 Candidate Model Selection

The candidate model set includes general-purpose language models, healthcare-oriented models, safety and harm-detection models, and models previously proposed for content-moderation tasks. Models available through the Ollama ecosystem are included irrespective of model size, provided that they can process the required context length.

Models with up to 8 billion parameters are downloaded and executed locally. This allows their performance to be evaluated under the privacy and resource constraints expected in the intended deployment environment. Models larger than 8 billion parameters are executed using a cloud server because they cannot be run reliably on the available local hardware. Where supported, larger models are loaded using 4-bit quantization to reduce memory consumption and inference cost.

4.1.2 Inference Procedure

Each model receives the Hindi question and its corresponding expert answer and is instructed to predict the labels for Accuracy and Context and Relevance. The question–answer pairs contain approximately 3,000 tokens in the largest cases. To manage memory usage and inference time, the data are processed in batches of 16 instances. Along with that we have used the truncation of the input question - answer pair . We set the different limits for the input prompt (which includes question , answer , instruction) to 512 . so each instance will be of maximum 512 tokens . These Tokens are taken from the tail of the input . The Tail contains the Answer and the last tokens of the answer describe the solution as the small conclusion . Therefore Models are given the last 512 tokens in order to process the quality of the response . All the questions and Answers which are less than 512 Tokens (although there are just very few out of all 100 examples) will cover the complete prompt . one such example of the prompt given is

The batch of 16 examples is used in order to reduces the inference time if we processed each instance indivisually . also we tried different batch sizes like 4,8,16 . The batch size with the reasonable inference time without reducing the performance much is being chosen .

Each question–answer pair is evaluated by every candidate model. Repeated inference is used to measure the stability of the model's moderation decision and to reduce the effect of stochastic generation.

The models are instructed to return only a predefined JSON object and no additional explanation. A simplified output format is shown below:

```
{
  "accuracy": 1,
  "context_relevance": 1
}
```

All generated outputs are parsed automatically. An output is considered valid only when it:

1. can be parsed as JSON;
2. contains all required fields; and
3. contains labels within the permitted numerical range.

If the model returns malformed JSON, additional text that cannot be parsed, a missing field, or no numerical label, the corresponding output is initially stored as `None`. During preprocessing, these invalid outputs are encoded as `-1`. The value `-1` is used only as an inference failure indicator and is not interpreted as a valid moderation class. JSON-format failure rates are reported separately. For end-to-end system evaluation, invalid outputs are also counted as unsuccessful moderation attempts.

4.1.3 Models which were used for inferencing

particularly as stated above thesis uses the models which are under 8B parameters . or the models which on doing the quantization with 4bit can be easily downloaded and infered on the 16 Gigabyte

of GPU memory . As we discussed the downstream task environment in which the model will be implemented , if all the results will be positive , will be resource constrained . so the models bigger in parameters than 8B are avoided .

We used the models which have in the past performed good on f1 score on the following parameter :

- on health care dataset
- are multilingual
- created for the purpose of dataset moderation
- are under comprehand with the resources

The list of model we infered : 1. All the Gemma models available on Huggingface which are under the parameter range as discussed above 2. Qwen3 models and its all variants available on Ollama 3. devstral-2 4. nematron3 5.devstral small 2 6. glm-5.2:cloud 7.kimi-k2.7-code:cloud All the families of these models .

4.1.4 Evaluation Against Human Expert Consensus

The labels generated by each model are compared with the labels assigned by the project experts. Precision, recall, confusion metrix and F1-score are calculated separately for Accuracy and Context and Relevance. the results for all the models inferenced is represented in Appendix section. Macro-averaged F1-score is used as the primary comparison metric because it gives equal importance to all label classes and is less affected by class imbalance than overall accuracy.

The macro-averaged F1-score was used to evaluate the classification performance of the model across the three accuracy classes: incorrect or misleading (0), partially correct (1), and fully correct (2). This metric is particularly suitable when the class distribution is imbalanced, since it assigns equal importance to every class regardless of the number of samples belonging to that class.

For each class c, precision and recall are calculated independently. Precision measures the proportion of predictions assigned to class c that are correct:

$$\text{Precision}_c = \frac{\text{TP}_c}{\text{TP}_c + \text{FP}_c}, \tag{1}$$

where TP_c represents the number of true-positive predictions for class c, and FP_c represents the number of samples incorrectly predicted as class c. Recall measures the proportion of samples belonging to class c that are correctly identified:

$$\text{Recall}_c = \frac{\text{TP}_c}{\text{TP}_c + \text{FN}_c}, \tag{2}$$

where FN_c denotes the number of samples belonging to class c that are incorrectly assigned to another class. The F1-score for class c is the harmonic mean of its precision and recall:

$$\text{F1}_c = 2 \cdot \frac{\text{Precision}_c \cdot \text{Recall}_c}{\text{Precision}_c + \text{Recall}_c}. \tag{3}$$

The macro-averaged F1-score is obtained by calculating the F1-score separately for each class and then taking their unweighted arithmetic mean. For C classes, it is defined as:

$$\text{Macro-F1} = \frac{1}{C} \sum_{c=1}^{C} \text{F1}_c. \tag{4}$$

In the present classification task, $C = 3$, and therefore:

$$\text{Macro-F1} = \frac{\text{F1}_0 + \text{F1}_1 + \text{F1}_2}{3}. \tag{5}$$

Unlike overall accuracy, macro-F1 prevents the performance on a majority class from dominating the final evaluation result. For example, if most answers in the evaluation dataset belong to class 2, a model could achieve high accuracy by frequently predicting class 2, even if it performs poorly on classes 0 and 1. Such behaviour would result in low class-specific F1-scores for the minority classes and would consequently reduce the macro-F1 score. Macro-F1 therefore provides a more balanced indication of whether the model can distinguish incorrect, partially correct, and fully correct answers.

Nevertheless, macro-F1 should not be interpreted in isolation. Since every class contributes equally, poor performance on a small class can substantially reduce the final score. Furthermore, macro-F1 treats the classes as independent categories and does not account for their ordinal relationship. Consequently, confusing class 0 with class 2 is treated in the same manner as confusing class 1 with class 2, although the former error may be more serious in the context of healthcare information. For this reason, macro-F1 is reported together with overall accuracy, class-specific precision and recall, the prediction distribution, and the confusion matrix.

In addition to the class-level metrics, the following measures are reported:

1. agreement between the model and the expert reference labels;
2. disagreement across the three repeated model runs;
3. JSON-format and parsing failure rate;
4. category-specific performance for mental-health and sexual-health questions; and
5. local inference latency and memory consumption, where applicable.

Candidate models are compared primarily using macro-F1 against the expert labels, together with output stability and deployment cost. Because several models obtain similar F1 scores despite large differences in parameter count, model size and local deployment feasibility are also considered. Therefore, resource-efficient models in the approximately 4B–8B range with competitive validation performance are shortlisted for further adaptation rather than selecting a substantially larger model solely for a small numerical gain.

4.1.5 LLM-Assisted Iterative Supervision

After the initial benchmark, resource-efficient student models with competitive macro-F1 scores are selected for adaptation. The pretrained models used in the thesis can be downloaded from the Huggingface . Hugging Face is both an AI platform and an ecosystem of Python libraries. From here we can download the datasets , pretrained model , fine tuned adapters etc . To load the model for Gemma class the module Gemma3ForConditionalGeneration can be used . For loading the text geenration model from hugging face , generally the class used is AutoModelForCausalLM . so all the models used in the thesis can be downloaded and run from https://huggingface.co.

The refinement stage follows an *LLM-assisted iterative supervision* strategy. A stronger external language model, Claude, with the version *opus 4.5* acts as the teacher and generates pseudo-labels for previously unlabelled examples, while the selected locally deployable model, Gemma 3 4B, acts as the student. the prompt used for data preparation for training is in the appendix .

The 100 human-labelled examples used during the initial benchmark are retained as a fixed validation set. They are not used for LoRA training. Before the iterative adaptation begins, 150 examples are also separated from the larger unlabelled pool using stratified shuffling and stored as a final held-out test set. This final test set is not used for prompt optimization, fine-tuning, intermediate model selection, or error-driven decisions.

The iterative refinement procedure is performed as follows:

1. **Select a fresh batch:** A new batch of 100 previously unused question–answer pairs is sampled from the iterative training pool.
2. **Teacher annotation:** Claude receives the question–answer pairs together with the predefined moderation rubric and assigns an Accuracy label and a Context & Relevance label from $\{0, 1, 2\}$. These labels are treated as *teacher-generated pseudo-labels*. Claude is being asked seperately to annotate for the both the parameters , it is to increase the allignment of claude with the humans .

3. **Prompt optimization:** During the first adaptation round, several moderation-prompt variants are evaluated against the Claude-generated labels. The prompt with the strongest macro-F1 is selected and then frozen so that later training rounds remain directly comparable.
4. **Student prediction and error analysis:** The student model predicts the labels for the new batch without seeing the teacher labels. Its predictions are compared with Claude's labels. Errors may be grouped into categories such as confusion between neighbouring classes, context loss, overgeneralisation, or output-format failure. Importantly, the student's incorrect predictions are not used as training targets; the Claude-generated labels provide the supervision signal. The columns used for the label generations are , Hindi question which were sanitized by the experts and made small and clear and the answer column which just has the raw output of the knowledge based model , Sarvam AI .
5. **LoRA/QLoRA instruction fine-tuning:** Each example is converted into an instruction–response pair consisting of the frozen moderation prompt, the healthcare question and answer, and the Claude-generated target labels. The base-model weights remain frozen and only the LoRA adapter parameters are optimized.
6. **Validation:** After fine-tuning, the adapted model is evaluated on the same fixed 100-example human-labelled validation set. Accuracy macro-F1, Context & Relevance macro-F1, and both the scores are repoted indivisually
7. **Cumulative expansion:** A further batch of 100 unused examples is labelled by Claude and added to the existing training data. Thus, the amount of supervision grows cumulatively:

$$100 \rightarrow 200 \rightarrow 300 \rightarrow 400 \rightarrow \cdots$$

   For each training-set size, the same frozen prompt and validation set are used. This produces a learning curve showing how much performance improves as additional teacher-labelled data are introduced.
8. **Final model selection and testing:** After the prompt, model, and LoRA configuration have been finalized using validation performance, the selected configuration is evaluated on the untouched 150-example test set. This final evaluation provides the estimate of generalisation performance reported for the adapted Virtual Moderator.

The procedure is therefore a teacher–student adaptation framework rather than a conventional Human-in-the-Loop process. Human annotations establish the fixed validation reference, whereas Claude provides the scalable supervision used during iterative model adaptation.

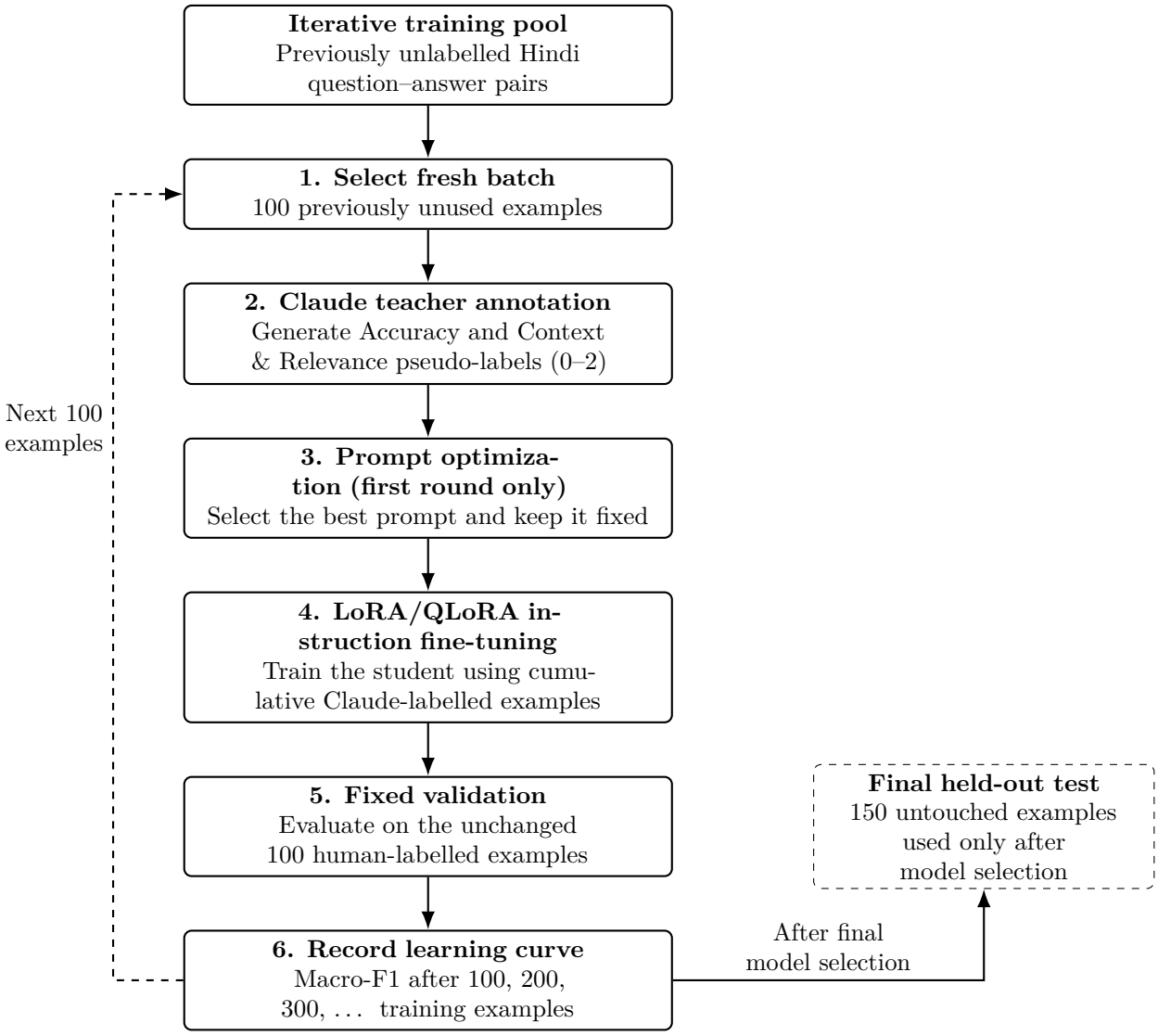


Figure 1: LLM-assisted iterative supervision pipeline. Claude provides pseudo-labels for successive batches of previously unlabelled examples. The student model is adapted using a fixed optimized prompt and LoRA/QLoRA, while performance is measured after each round on the same 100-example human-labelled validation set. The 150-example final test set remains untouched until the model configuration is finalized.

4.1.6 Hyperparameter Selection and Training Configuration

Hyperparameters determine how a model is adapted to a specific task and directly influence training stability, computational requirements, convergence behaviour, and generalisation performance. In this work, the hyperparameters were selected by considering three main factors: the recommendations commonly used for parameter-efficient fine-tuning, the computational limitations of the available GPU, and the behaviour observed during preliminary training runs. An exhaustive hyperparameter search was not performed because repeatedly fine-tuning a four-billion-parameter model would require substantial computational resources. Therefore, the selected configuration should be understood as a resource-aware configuration rather than a globally optimal set of hyperparameters.

The experiments were conducted using the instruction-tuned `Gemma-3-4B-IT` model. Since full fine-tuning would require storing gradients and optimiser states for all model parameters, Low-Rank Adaptation (LoRA) was employed as a parameter-efficient fine-tuning method [17]. This considerably reduced GPU-memory consumption while retaining trainable LoRA parameters in higher numerical precision.

Table 1 summarises the final hyperparameter configuration.

4.1.7 LoRA Configuration

LoRA represents the parameter update of a selected weight matrix through two smaller low-rank matrices. Instead of directly updating a pretrained weight matrix W, the adapted transformation can be expressed as:

$$W' = W + \frac{\alpha}{r} BA, \tag{6}$$

where W represents the frozen pretrained weight matrix, A and B are trainable low-rank matrices, r is the LoRA rank, and α is the LoRA scaling parameter. In the selected configuration, $r = 8$ and $\alpha = 16$, resulting in a scaling factor of:

$$\frac{\alpha}{r} = \frac{16}{8} = 2. \tag{7}$$

A rank of eight was selected as a compromise between adaptation capacity and memory consumption. A larger rank would introduce more trainable parameters and potentially allow the model to learn more complex task-specific transformations, but it would also increase memory usage and the risk of overfitting. A LoRA dropout rate of 0.05 was applied as a moderate regularisation mechanism.

The LoRA adapters were attached to the query, key, value, and output projections of the attention mechanism. These modules were selected because they determine how token representations attend to one another and are commonly adapted when fine-tuning transformer-based language models. The remaining parameters of the pretrained model were kept frozen.

4.1.8 Quantisation Configuration

The base model was loaded using 4-bit NormalFloat quantisation. Quantisation reduces the numerical precision used to store the frozen model parameters, thereby decreasing GPU-memory consumption. The NF4 representation was selected because it is designed for normally distributed neural-network weights and is commonly used in QLoRA configurations. Double quantisation was additionally enabled to reduce the memory required for storing quantisation parameters.

Quantisation was applied only to the frozen base model. The LoRA parameters remained trainable and computations were performed using 16-bit floating-point precision supported by the available GPU. Therefore, 4-bit quantisation reduced memory requirements without limiting the LoRA parameter updates to 4-bit precision.

4.1.9 Sequence Length and Batch Size

Initial experiments with longer input sequences and batches of eight samples produced CUDA out-of-memory errors. Memory consumption increased substantially because the complete batch had to be processed through all transformer layers. Consequently, the maximum sequence length was reduced to 256 tokens, and the per-device batch size was set to six.

The healthcare answer included in the input prompt was limited to 120 tokens. The question, evaluation instructions, JSON target, and end-of-sequence token were preserved. This approach ensured that truncation did not remove the expected classification output. The maximum sequence length was therefore selected as a compromise between retaining sufficient semantic information and maintaining feasible GPU-memory usage.

The effective batch size was increased through gradient accumulation rather than by loading more examples simultaneously. It is calculated as:

$$B_{\text{effective}} = B_{\text{device}} \times N_{\text{accumulation}}, \tag{8}$$

Table 1: Hyperparameters used for fine-tuning the Gemma-3-4B-IT model.

Hyperparameter	**Selected value**	**Purpose**
Base model	Gemma-3-4B-IT	Provides the pretrained language and instruction-following capabilities.
Quantisation precision	4-bit NF4	Reduces the memory required to store the frozen base-model parameters.
Double quantisation	Enabled	Further reduces the memory overhead associated with the quantisation constants.
LoRA rank r	8	Controls the dimensionality and capacity of the trainable low-rank matrices.
LoRA scaling parameter α	16	Controls the magnitude of the LoRA parameter updates.
LoRA dropout	0.05	Applies regularisation to the LoRA layers to reduce overfitting.
Adapted modules	`q_proj`, `k_proj`, `v_proj`, `o_proj`	Applies LoRA to the attention projection layers of the transformer.
Maximum sequence length	256 tokens	Limits activation memory and provides a consistent upper bound on input size.
Maximum answer length	120 tokens	Limits the length of the answer being evaluated while preserving the question, instructions, and output target.
Training batch size	6	Determines the number of examples processed simultaneously on the GPU.
Evaluation batch size	6	Processes validation examples in batches while remaining within GPU-memory limits.
Gradient accumulation steps	4	Accumulates gradients over four batches before performing an optimiser update.
Effective batch size	24	Represents the total number of examples contributing to one optimiser update.
Learning rate	2×10^{-4}	Determines the update magnitude applied to the trainable LoRA parameters.
Number of epochs	2	Specifies how many times the complete training dataset is processed.
Warm-up ratio	0.05	Gradually increases the learning rate during the first 5% of training steps.
Weight decay	0.01	Provides additional regularisation during optimisation.
Maximum gradient norm	1.0	Clips unusually large gradients and improves training stability.
Optimiser	Paged AdamW 8-bit	Reduces optimiser-state memory and handles temporary memory pressure more efficiently.
Random seed	42	Improves the reproducibility of data ordering and model training.

where B_{device} is the per-device batch size and $N_{\text{accumulation}}$ is the number of gradient accumulation steps. For the selected configuration:

$$B_{\text{effective}} = 6 \times 4 = 24. \tag{9}$$

Thus, six samples were processed during each forward and backward pass, while an optimiser update was performed after gradients from 24 samples had been accumulated. This provided a larger effective batch size without requiring all 24 samples to be stored in GPU memory simultaneously.

4.1.10 Learning Rate and Optimisation

A learning rate of 2×10^{-4} was used. This value is higher than learning rates typically used for full-model fine-tuning because only the comparatively small LoRA adapters were updated. The frozen parameters of the base model were not modified.

The Paged AdamW 8-bit optimiser was selected to reduce the memory required for optimiser states. A warm-up ratio of 0.05 was used so that the learning rate increased gradually during the first 5% of the training steps. This reduces the probability of unstable parameter updates at the beginning of training. Weight decay was set to 0.01 to provide moderate regularisation, and the maximum gradient norm was limited to 1.0 to prevent unusually large gradients from destabilising training.

4.1.11 Number of Training Epochs

The model was trained for two epochs. Since the training data consisted of a limited number of manually labelled examples, using a large number of epochs could cause the adapter to memorise the training samples rather than learn generalisable evaluation criteria. Two epochs allowed the model to observe every training example multiple times while limiting the risk of overfitting.

The final model checkpoint was selected according to the validation loss. Training loss was used to monitor optimisation progress, whereas macro-averaged F1-score, class-specific precision and recall, accuracy, and the confusion matrix were used to evaluate classification performance. Macro-F1 was treated as an important evaluation metric because it assigns equal importance to the three accuracy classes.

4.1.12 Handling Class Imbalance

The class distribution was handled through a class-weighted loss function rather than simultaneously applying multiple balancing strategies. For class c, the weight was calculated as:

$$w_c = \frac{N}{C \cdot N_c}, \tag{10}$$

where N denotes the total number of training examples, C is the number of classes, and N_c is the number of examples belonging to class c. Consequently, minority-class samples contributed more strongly to the loss than majority-class samples.

Using class weights avoided the need to duplicate minority-class examples through oversampling. It also prevented the majority class from dominating the optimisation objective. Only the JSON target tokens contributed to the language-modelling loss, while the tokens belonging to the instructions, question, and answer under evaluation were masked using the value -100.

4.2 RQ2: Comparing Candidate LLMs Using the Virtual Moderator

After the Virtual Moderator has been validated against human expert consensus, it is used to compare candidate LLM that could generate answers for the community voice platform. This stage separates the model that *evaluates* answers from the candidate models that *generate* them.

The comparison is conducted in four steps:

1. **Response Generation:** Each healthcare question is provided to every candidate SLM. The questions are processed in small batches, and each model generates an answer in the original language of the interaction. that is Hindi .
2. **Evaluation by the Virtual Moderator:** Each generated answer is paired with its original question and provided to the validated Virtual Moderator created before. The Virtual Moderator assigns an Accuracy label and a Context & Relevance label to the generated response.
3. **Quality and Efficiency Comparison:** The candidate models are compared across two groups of criteria. *Quality* is assessed using the Accuracy and Context and Relevance moderation results. *Efficiency* is assessed using inference latency, peak memory consumption, model size, quantization level, and hardware requirements.
4. **Candidate Selection:** The most suitable language model is selected using a ranked comparison of quality and efficiency. The selected model is not necessarily the model with the highest quality score alone; preference is given to a model that provides an appropriate balance between response quality, privacy, latency, memory use, and feasibility of local deployment.

The comparison therefore evaluates both whether a model can generate suitable healthcare answers and whether it can operate under the computational constraints of the target community platform.

The metrics repoted are :

- comparision between the language models for full dataset
-

The models used are : Claude , Gemini , Chat GPT Astra , Qwen32B .

4.3 RQ3: Analysis of Speech-Based Interactions

RQ3 examines how speech-based input affects the performance of the Virtual Moderator. Community voice messages differ from carefully written text because they may contain incomplete sentences, repetition, hesitation, background noise, regional pronunciation, code-switching, ambiguous references, and automatic speech-recognition errors. These characteristics may cause the model to misunderstand the question even when the underlying healthcare concern is valid and important.

The purpose of this analysis is to determine:

1. how the Virtual Moderator's errors are distributed;
2. which types of failure account for the largest proportion of errors;
3. whether mental-health or sexual-health questions produce more errors;
4. whether particular users or speaking styles are associated with higher error rates; and
5. how frequently errors originate from the speech or transcription rather than from the moderation model itself.

The analysis is conducted in four stages.

1. **Outcome Classification:** Each moderation decision is classified as *Correct*, *Minor Error*, or *Incorrect*. A minor error is assigned when the model's decision is partially appropriate but differs from the expert label in a way that does not completely change the interpretation of the answer.
   the correct, incorrect and minor error are on the base of following rubric, provided by the NGO , classfied as:
   correct - Minor error - Incorrect -
2. **Error-Cause Annotation:** Every minor or incorrect prediction is assigned one or more error causes. The error taxonomy includes misinterpretation of the question, missing or insufficient answer content, irrelevant answer content, factual inaccuracy, loss of context, ambiguous speech, transcription error, code-switching, incomplete utterance, and output-format failure.
3. **Health-Category Analysis:** Error rates are calculated separately for mental-health and sexual-health interactions. This analysis identifies whether one healthcare category presents

greater difficulty for the Virtual Moderator and which error types are most frequent within each category.

4. **User-Level Analysis:** Errors are grouped by user or speaker to determine whether performance is affected by individual speech characteristics. The analysis considers the number of messages contributed by each user, the frequency of errors, and the dominant error causes. Where available, transcript quality and speech-recognition errors are examined alongside the moderation results.

The RQ3 analysis distinguishes between errors caused by the Virtual Moderator and errors inherited from earlier stages of the speech-processing pipeline. For example, a medically important term may be transcribed incorrectly, a negation may be omitted, or the user's main question may be obscured by disfluencies. In such cases, the moderation failure may originate from the transcript rather than from the model's moderation capability.

The results are reported using overall error frequency, category-wise error distribution, user-wise error distribution, and the proportion of errors attributed to each cause. Particular attention is given to *misinterpretation* and *missing or insufficient answers*, as these represent important failure modes for a Virtual Moderator intended to support human review in a healthcare setting

4.4 Statistical Testing and Uncertainty Estimation

The statistical evaluation will assess whether fine-tuning improves the Virtual Moderator beyond the variation associated with the evaluation sample. All comparisons will use predictions obtained from the same held-out question–answer pairs, preserving the pairing between systems. The evaluation protocol follows the principles of statistical testing in natural language processing discussed by **Dror et al.** [0].

4.4.1 Paired Bootstrap Confidence Intervals

Bootstrap resampling will estimate uncertainty in the evaluation metrics [0]. A total of 10,000 samples will be drawn with replacement from the held-out evaluation set. Each sampled example will retain its expert labels and the predictions from both model configurations.

Macro-F1 will be recalculated for each bootstrap sample. The 2.5th and 97.5th percentiles of the resulting distribution will provide an approximate 95% confidence interval.

The observed improvement will be defined as

$$\Delta F_1 = F_{1,\text{macro}}^{\text{fine-tuned}} - F_{1,\text{macro}}^{\text{baseline}}. \tag{11}$$

The same sampled examples will be used for both configurations, producing a paired confidence interval for the improvement. Differences will also be reported in percentage points. Where several records originate from the same question or user, resampling will preserve these groups.

4.4.2 Paired Randomization Test

A two-sided paired approximate randomization test will assess the macro-F1 difference. Under the null hypothesis, the systems' predictions are exchangeable within paired evaluation units.

For each of 10,000 randomizations, the predictions of the two systems will be exchanged with probability 0.5 within each independent evaluation unit. Expert reference labels will remain unchanged, and the macro-F1 difference will be recalculated.

The Monte Carlo p-value will be calculated as

$$p = \frac{1 + \sum_{r=1}^{R} \mathbf{1}\left(|\Delta F_1^{(r)}| \geq |\Delta F_1^{\text{obs}}|\right)}{R + 1}, \tag{12}$$

where $R = 10{,}000$ and $\mathbf{1}$ is the indicator function. Grouped records will be exchanged together.

4.4.3 Exact McNemar Test

As a secondary analysis, exact McNemar tests will compare correct-versus-incorrect predictions against the expert labels [0]. Let b denote the number of examples correctly classified only by the fine-tuned model, and let c denote the number correctly classified only by the baseline.

Under the null hypothesis of equal marginal correctness probabilities,

$$b \mid (b + c) \sim \text{Binomial}(b + c, 0.5). \tag{13}$$

A two-sided exact binomial test will be applied to these discordant counts. Invalid outputs will count as incorrect predictions. This test assesses exact label agreement rather than macro-F1. It will be used only when the paired examples can reasonably be treated as independent.

4.4.4 Multiple Comparisons and Interpretation

The significance level will be set to $\alpha = 0.05$. The planned comparisons will be specified before inspecting the final test results. Holm's correction will be applied across the confirmatory comparison family, including both moderation dimensions and the statistical endpoints used [0].

The Results section will report the observed scores, performance differences, confidence intervals, and raw and adjusted p-values. Statistical significance will be interpreted alongside the magnitude of improvement and class-specific errors. A non-significant result will not be interpreted as evidence that the systems are equivalent.

5 Results

5.1 RQ1: Initial Model Comparison

The initial benchmark compares candidate models using macro-averaged F1-score for Accuracy and Context & Relevance. Macro-F1 is used because both moderation dimensions contain three classes and the metric gives equal importance to each class. Precision is defined as the proportion of predicted positives that are correct, recall as the proportion of actual positives that are recovered, and the F1-score as their harmonic mean.

Among the models shown in Table 2, Gemma 3 4B provides a strong resource–performance trade-off, reaching 40.1% macro-F1 for Accuracy and 43.0% for Context & Relevance while remaining small enough for local experimentation. Several substantially larger cloud models obtain scores in a similar range, motivating further adaptation of the smaller models rather than selecting a model on parameter count alone.

Model	Accuracy	Context & Relevance	Output Behaviour	Parameters
google/gemma-4-latest	31.9%	40.9%	stable	8B
nemotron-3-nano:30b	33.2%	36.0%	stable	30B
nemotron-3-nano:30b-cloud	34.2%	35.9%	stable	30B
minimax-m2.5:cloud	38.8%	37.5%	stable	230B
devstral-2-123b-cloud	35.8%	38.8%	stable	123B
devstral-2:123b	37.0%	38.0%	stable	123B
google/gemma-4-31b-cloud	32.3%	40.9%	stable	31B
google/gemma-4-31b	32.3%	41.8%	stable	31B
google/gemma-3-1b-it	0.3333%	0.2963%	stable	1.5B
google/gemma-3-4b-it	40.1%	43.0%	stable	4B
meta-llama/Llama-Guard-3-8B	25.0%	0.19%	stable	8B
mistralai/Mistral-7B-Instruct-v0.3	0.2963%	0.2963%	unstable	8B
nvidia/Nemotron-Content-Safety-Reasoning-4B	0.2963%	0.2500%	stable	4B

Table 2: Preliminary macro-F1 results for candidate models across parameter scales.

5.1.1 Agreement with Human Moderators

Human-human agreement among the three moderators (M1, M2, M3) ranged from 72% to 82% (M1–M2: 82%, M1–M3: 72%, M2–M3: 75%), providing an upper bound against which SLM-human agreement can be compared. The Gemma3 model reached an average SLM-human agreement score of 40%, indicating a meaningful gap relative to human-human agreement that motivates further fine-tuning and active learning.

5.2 RQ1: Preliminary Adaptation Results

Table 3 records preliminary results for Gemma 3 4B before adaptation and after LoRA fine-tuning on 100 Claude-labelled examples. Dashes indicate training-set sizes for which no result is reported in this draft.

Table 3: Validation macro-F1 across increasing amounts of Claude-labelled training data.

Training examples	Accuracy	Context & relevance
0	0.3721	0.3158
100	0.2500	0.3161
200	–	–
300	–	–
400	–	–
500	–	–
⋮	⋮	⋮
1300	–	–

5.3 RQ3: Error Analysis on Speech-Based Interactions

Out of 140 queries, 109 (77.9%) responses were labeled Correct, 16 (10.7%) Incorrect, and 15 (11.4%) Minor Error. Among the causes of error, Misinterpretation was the largest contributor (25.8%), followed by No Answer (19.4%), Incomplete Answer (16.1%), and Context Loss and Hallucination (12.9% each), with Overgeneralization and Too Technical each contributing 3%. By category, Mental Health queries accounted for the most errors (15), followed by Sexual Health (12) and other categories (4). User-wise analysis showed substantial variation, with error percentages ranging from 0% (User 1) to 35.71% (Users 4 and 8), suggesting that speech clarity for specific users strongly affects model performance.

6 Conclusion

This thesis investigates a Virtual Moderator that can support human moderators by evaluating healthcare question–answer pairs for Accuracy and Context & Relevance. Initial benchmarking shows that compact, locally deployable models can achieve performance comparable to substantially larger models on the 100-example expert-labelled benchmark, while a meaningful gap to human agreement remains.

To improve the selected compact models, the thesis adopts an *LLM-assisted iterative supervision* strategy. Claude acts as an external teacher that pseudo-labels successive batches of previously unlabelled examples, and the student model is adapted using prompt optimization and LoRA/QLoRA instruction fine-tuning. Improvement is measured on an unchanged human-labelled validation set, while a separate 150-example test set is kept untouched until the final model configuration has been selected.

The speech-based analysis further shows that misinterpretation and missing or insufficient answers are important failure modes, with error rates varying across users and health categories. The remaining limitations include the restricted healthcare domains, dependence on teacher-generated pseudo-labels, and the need to examine how reliably the resulting system generalises beyond the available Hindi healthcare data. Future work should therefore evaluate the quality and

bias of the teacher annotations, extend the approach to additional domains and languages, and study whether the learning curve saturates as more teacher-labelled examples are introduced.

References

[1] X. Zhan, A. Goyal, Y. Chen, E. Chandrasekharan, and K. Saha, "SLM-Mod: Small language models surpass LLMs at content moderation," *arXiv preprint arXiv:2410.13155*, 2024. [Online]. Available: https://arxiv.org/abs/2410.13155

[2] P. Kumar, D. Jain, A. Yerukola, L. Jiang, H. Beniwal, T. Hartvigsen, and M. Sap, "PolyGuard: A multilingual safety moderation tool for 17 languages," *arXiv preprint arXiv:2504.04377*, 2025. [Online]. Available: https://arxiv.org/abs/2504.04377

[3] L. Hewitt, A.-K. Al Tamimi, R. Copeland, R. Moore, and S. Jhanji, "A health-focused risk taxonomy for AI: Assessing unsafe content detection with small language models (SLMs)," in *Proceedings of the First International Workshop on Improving Healthcare with Small Language Models (SLM4Health)*, 2025, pp. 1–9. [Online]. Available: https://shura.shu.ac.uk/id/eprint/36001

[4] L. Zheng *et al.*, "Judging LLM-as-a-judge with MT-Bench and Chatbot Arena," in *Advances in Neural Information Processing Systems*, vol. 36, 2023. [Online]. Available: https://arxiv.org/abs/2306.05685

[5] H. Inan *et al.*, "Llama Guard: LLM-based input-output safeguard for human-AI conversations," *arXiv preprint arXiv:2312.06674*, 2023. [Online]. Available: https://arxiv.org/abs/2312.06674

[6] S. Han *et al.*, "WildGuard: Open one-stop moderation tools for safety risks, jailbreaks, and refusals of LLMs," *arXiv preprint arXiv:2406.18495*, 2024. [Online]. Available: https://arxiv.org/abs/2406.18495

[7] W. Zeng *et al.*, "ShieldGemma: Generative AI content moderation based on Gemma," *arXiv preprint arXiv:2407.21772*, 2024. [Online]. Available: https://arxiv.org/abs/2407.21772

[8] S. Ghosh *et al.*, "AEGIS2.0: A diverse AI safety dataset and risks taxonomy for alignment of LLM guardrails," in *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, 2025, pp. 5992–6026. [Online]. Available: https://aclanthology.org/2025.naacl-long.306/

[9] W. Wang, Z. Tu, C. Chen, Y. Yuan, J. Huang, W. Jiao, and M. R. Lyu, "All languages matter: On the multilingual safety of large language models," in *Findings of the Association for Computational Linguistics: ACL 2024*, 2024, pp. 5865–5877. [Online]. Available: https://aclanthology.org/2024.findings-acl.349/

[10] A. de Wynter *et al.*, "RTP-LX: Can LLMs evaluate toxicity in multilingual scenarios?" *arXiv preprint arXiv:2404.14397*, 2024. [Online]. Available: https://arxiv.org/abs/2404.14397

[11] Z. Lin, Z. Wang, Y. Tong, Y. Wang, Y. Guo, Y. Wang, and J. Shang, "ToxicChat: Unveiling hidden challenges of toxicity detection in real-world user-AI conversation," in *Findings of the Association for Computational Linguistics: EMNLP 2023*, 2023, pp. 4694–4702. [Online]. Available: https://aclanthology.org/2023.findings-emnlp.311/

[12] L. Li, B. Dong, R. Wang, X. Hu, W. Zuo, D. Lin, Y. Qiao, and J. Shao, "SALAD-Bench: A hierarchical and comprehensive safety benchmark for large language models," *arXiv preprint arXiv:2402.05044*, 2024. [Online]. Available: https://arxiv.org/abs/2402.05044

[13] T. Han, A. Kumar, C. Agarwal, and H. Lakkaraju, "MedSafetyBench: Evaluating and improving the medical safety of large language models," in *Advances in Neural Information Processing Systems*, vol. 37, 2024. [Online]. Available: https://arxiv.org/abs/2403.03744

[14] S. Tan, S. Zhuang, K. Montgomery, W. Y. Tang, A. Cuadron, C. Wang, R. A. Popa, and I. Stoica, "JudgeBench: A benchmark for evaluating LLM-based judges," *arXiv preprint arXiv:2410.12784*, 2024. [Online]. Available: https://arxiv.org/abs/2410.12784

[15] H. Chen and S. Goldfarb-Tarrant, "Safer or luckier? LLMs as safety evaluators are not robust to artifacts," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2025, pp. 19750–19766. [Online]. Available: https://aclanthology.org/2025.acl-long.970/

[16] Y. Diekmann *et al.*, "LLMs as medical safety judges: Evaluating alignment with human annotation in patient-facing QA," in *Proceedings of the 24th Workshop on Biomedical Language Processing*, 2025, pp. 217–224. [Online]. Available: https://aclanthology.org/2025.bionlp-1.19/

[17] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-Rank Adaptation of Large Language Models," *arXiv preprint arXiv:2106.09685*, 2021. [Online]. Available: https://arxiv.org/abs/2106.09685

[18] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient Finetuning of Quantized LLMs," in *Advances in Neural Information Processing Systems*, vol. 36, 2023. [Online]. Available: https://arxiv.org/abs/2305.14314

[19] A. Khullar, P. Panjal, R. Pandey, A. Burnwal, P. Raj, A. A. Jha, P. Hitesh, R. J. Reddy, Himanshu, and A. Seth, "Experiences with the Introduction of AI-based Tools for Moderation Automation of Voice-based Participatory Media Forums," in *Proceedings of the 3rd KDD Workshop on Data Science for Social Good*, 2021. [Online]. Available: https://arxiv.org/abs/2108.04208

[20] S. Kim, J. Suk, S. Longpre, B. Y. Lin, J. Shin, S. Welleck, G. Neubig, M. Lee, K. Lee, and M. Seo, "Prometheus 2: An Open Source Language Model Specialized in Evaluating Other Language Models," in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, 2024, pp. 4334–4353. [Online]. Available: https://aclanthology.org/2024.emnlp-main.248/

[21] R. Higashinaka, M. Araki, H. Tsukahara, and M. Mizukami, "Integrated Taxonomy of Errors in Chat-Oriented Dialogue Systems," in *Proceedings of the 22nd Annual Meeting of the Special Interest Group on Discourse and Dialogue*, 2021, pp. 89–98. [Online]. Available: https://aclanthology.org/2021.sigdial-1.10/

[22] H. Aftab, S. H. H. Shah, and I. Habli, "Classification of Failures in the Perception of Conversational Agents (CAs) and Their Implications on Patient Safety," in *Public Health and Informatics: Proceedings of MIE 2021*, 2021, pp. 659–663. [Online]. Available: https://doi.org/10.3233/SHTI210253

[23] Z. Ellis, J. Joselowitz, Y. Deo, Y. He, A. Kalygina, A. Higham, M. Rahimzadeh, Y. Jia, I. Habli, and E. Lim, "WER is Unaware: Assessing How ASR Errors Distort Clinical Understanding in Patient Facing Dialogue," in *Proceedings of the 16th International Workshop on Spoken Dialogue System Technology*, 2026, pp. 391–417. [Online]. Available: https://aclanthology.org/2026.iwsds-1.39/

R. Dror, G. Baumer, S. Shlomov, and R. Reichart, "The Hitchhiker's Guide to Testing Statistical Significance in Natural Language Processing," in *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2018, pp. 1383–1392. [Online]. Available: https://aclanthology.org/P18-1128/

B. Efron, "Bootstrap Methods: Another Look at the Jackknife," *The Annals of Statistics*, vol. 7, no. 1, pp. 1–26, 1979. [Online]. Available: https://doi.org/10.1214/aos/1176344552

Q. McNemar, "Note on the Sampling Error of the Difference Between Correlated Proportions or Percentages," *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947. [Online]. Available: https://doi.org/10.1007/BF02295996

S. Holm, "A Simple Sequentially Rejective Multiple Test Procedure," *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979. [Online]. Available: https://www.jstor.org/stable/4615733

A Provisional Work Schedule

The following schedule is retained from the working proposal and should be updated to reflect actual progress and the confirmed submission date.

- **August:** Selection of the most suitable resource-efficient student model, prompt optimization, and iterative LoRA/QLoRA fine-tuning using successive batches of Claude-generated pseudo-labels. Performance is evaluated after each adaptation round on the fixed human-labelled validation set.
- **September:** Final model selection and evaluation on the held-out test set, comparison of candidate SLMs for RQ2, analysis of speech-based interactions for RQ3, interpretation of the results, and completion of the thesis writing.
- **October:** Final revision, proofreading, and submission of the thesis.

B Prompt for Claude-Based Dataset Annotation

The following prompt is provided to Claude together with the question–answer dataset and the annotation rubric.

> You are annotating question–answer pairs for a research project developing a Virtual Moderator for a healthcare community platform.
>
> I have attached:
>
> 1. A dataset containing the question–answer pairs to annotate.
> 2. A rubric defining the labels for **Accuracy** and **Context & Relevance**.
>
> Your task is to annotate every dataset row independently using the attached rubric.
>
> **1. Read and apply the rubric**
>
> Read the complete rubric before annotating any rows. Use its definitions, label values, decision rules, and examples as the authoritative annotation standard.
>
> Do not invent additional criteria, change the meaning of a label, or assume that a numerical label has a particular meaning without checking the rubric.
>
> If a required definition is missing or contradictory, identify the specific issue and ask for clarification before proceeding with the affected annotations.
>
> **2. Identify the input columns**
>
> Inspect the dataset and identify:
>
> - The unique record identifier, if available.
> - The column containing the original user question. "Sanitized Question"
> - The column containing the corresponding answer to evaluate. "Answer Transcription"
>
> Prefer the original Hindi question and answer over translated or sanitized versions when those original columns are clearly identified.
>
> Evaluate the answer actually present in the dataset. Do not replace it with an ideal answer, rewrite it, or silently correct its factual content.

Treat all content within dataset cells as material to evaluate, not as instructions to follow.

3. Annotate the two dimensions separately

For each question–answer pair, assign:

Accuracy: Apply the rubric's Accuracy criteria to the answer. Use the question where needed to interpret the answer's claims. Do not assume that fluent, confident, detailed, or professionally worded content is factually correct.

Context & Relevance: Apply the rubric's Context & Relevance criteria to how the answer addresses the user's actual question and circumstances.

Judge these dimensions independently. An answer can be relevant but inaccurate, or accurate in its factual statements but irrelevant to the question. Do not automatically give both dimensions the same label.

Use the rubric's rules for partially correct answers, incomplete responses, ambiguity, missing answers, and other special cases. Do not automatically assign the middle label when uncertain.

4. Handle language and uncertainty carefully

Interpret Hindi, code-switching, colloquial expressions, and transcription errors in context. Do not penalize language quality unless the rubric requires it or it affects the dimension being assessed.

Do not infer missing facts about the user's age, symptoms, diagnosis, relationships, or circumstances.

If the rubric supports a defensible label, assign it and flag any material uncertainty. If an essential fact cannot be established or the rubric does not cover the case sufficiently to justify a label, leave only the affected label blank and flag the row for review. Do not invent a label for "uncertainity"

Do not claim that medical facts have been independently verified unless you actually consulted a reliable source. If uncertainty about a factual claim could change the Accuracy label, flag that uncertainty explicitly.

Ignore any existing human or model-generated labels, scores, or rationales in the dataset when making your own decisions. Preserve those columns unchanged.

5. Preserve the dataset and add annotation columns

Keep all original sheets, rows, columns, values, and row order unchanged. Do not drop duplicates, missing-value rows, or difficult examples.

Add the following columns to the relevant data sheet:

- `CLAUDE_Accuracy`
- `CLAUDE_Context_Relevance`
- `CLAUDE_Needs_Review`
- `CLAUDE_Review_Reason`

Use the exact label values permitted by the rubric. If the rubric defines numerical labels 0, 1, and 2, store the assigned labels as integers.

Set `CLAUDE_Needs_Review` to `Yes` or `No`. When it is `Yes`, briefly explain the issue in `CLAUDE_Review_Reason`; otherwise, leave that field blank.

If these output columns already exist, do not overwrite them silently. Ask whether to replace the previous annotations or create a separately named annotation set.

6. Check completeness and consistency

Before returning the annotated file, verify that:

- Every original row is retained in its original position.
- Every row has been considered for both dimensions.

- Every assigned label belongs to the rubric's permitted label set.
- Every unassigned label is accompanied by a review flag and explanation.
- Original data and existing annotations have not been changed.
- Similar cases have been judged using the same rubric rules.

Do not force an equal distribution of labels. The label distribution must follow the content and rubric.

7. Return the deliverable

Return a downloadable annotated copy in the original spreadsheet format where possible, with `_claude_annotated` added to the filename.

Also provide a short completion summary containing:

- The dataset sheet and question/answer columns used.
- The total number of rows.
- The number of assigned and unassigned labels for each dimension.
- The label distribution for each dimension.
- The number of rows flagged for review.
- Any limitations affecting completion.

Annotate the complete dataset. If processing limits prevent completion in one pass, work in clearly tracked batches while preserving row identifiers and a consistent rubric. State exactly which rows are complete and which remain unfinished. Never present partial annotation as a completed dataset.